You are working inside the project:

```
/media/ambijat/SOPRANO2/GPT_workflow/COMTRADE
```

This project is a migration and reconstruction of an old successful UN Comtrade R-based workflow, now broken because the legacy UN Comtrade API procedure is no longer usable. The goal is not to create a generic downloader. The goal is to recreate the ontology, logic, and output discipline of the old R7 project using modern Python and the current UN Comtrade API.

The project already contains an ontology report here:

```
/media/ambijat/SOPRANO2/GPT_workflow/COMTRADE/ontology/
UNCOMTRADE_R7_ONTOLOGICAL_PROJECT_REPORT.md
```

You must treat this report as the conceptual authority for the project. The Python implementation must explicitly preserve the old R7 logic:

```
download → dress / clean → transpose / wide panel
```

The old project used R scripts to query the legacy endpoint:

```
http://comtrade.un.org/api/get?
```

with old-style parameters such as:

```
px=HS
ps=year
r=reporter
p=partner
rg=1 / 2
cc=commodity code
fmt=csv
```

That old access route is no longer the correct procedure. Recreate the workflow using the modern UN Comtrade Python package / API logic, preferably `comtradeapicall`, using a local subscription key file.

The key must not be hard-coded inside the script. It must be read from:

```
/media/ambijat/SOPRANO2/GPT_workflow/COMTRADE/secrets/
comtrade_primary_key.txt
```

The file contains only the Comtrade subscription key as plain text.

The project should use this structure:

```
COMTRADE/
├── ontology/
│   └── UNCOMTRADE_R7_ONTOLOGICAL_PROJECT_REPORT.md
├── config/
│   └── panel_config.json
├── secrets/
│   └── comtrade_primary_key.txt
├── reference/
│   ├── HSag2_partner.csv
│   ├── HSag4_partner.csv
│   ├── HSag6_partner.csv
│   └── partner_list.csv
├── comtrade_panel_builder.py
├── comtrade_panel_output/
│   ├── raw/
│   ├── processed/
│   └── wide/
└── .gitignore
```

Create missing folders if they do not already exist.

Also ensure `.gitignore` contains:

```
secrets/
*.key
*.env
__pycache__/
*.pyc
comtrade_panel_output/raw/
```

Do not delete existing ontology or reference files.

Now implement or rewrite:

```
comtrade_panel_builder.py
```

The script must be a robust, self-contained Python builder that performs the following:

1. Reads the ontology path and refers to it in a top-level docstring.
2. Reads configuration from:

```
config/panel_config.json
```

1. Reads the Comtrade subscription key from:

```
secrets/comtrade_primary_key.txt
```

1. Uses modern UN Comtrade API logic, preferably the official `comtradeapicall` package.
2. Fetches import and export data for the configured reporter, years, partner, classification, and commodity codes.
3. Uses batching for commodity codes so that API calls remain manageable.
4. Saves raw yearly CSV files under:

```
comtrade_panel_output/raw/import/
comtrade_panel_output/raw/export/
```

1. Creates processed long-format cleaned panels under:

```
comtrade_panel_output/processed/
```

1. Creates wide / transposed analytical tables under:

```
comtrade_panel_output/wide/
```

1. Preserves the old R7 logic of having a complete partner × commodity × year panel where missing trade values are filled with zero.
2. Writes a validation summary report after each run under:

```
comtrade_panel_output/validation_report.txt
```

1. Prints clear progress messages to the terminal, such as:

```
Starting UN Comtrade panel builder
Using subscribed getFinalData API
Fetching import | year=2009 | batch=1 | codes=20
Saved raw yearly file: ...
Creating processed long panel
Creating wide panel
Validation complete
```

Use the following mapping from old R7 flow logic to modern flow logic:

```
old rg=1 → import → modern flowCode="M"
old rg=2 → export → modern flowCode="X"
```

Use annual HS data:

```
frequency = "A"
classification = "HS"
```

Use reporter code from config. India is usually:

```
reporter_code = "699"
```

The initial config should be small and testable. Create this file if missing:

```
config/panel_config.json
```

with content similar to:

```
{
  "reporter_code": "699",
  "reporter_name": "India",
  "years": [2009, 2010],
  "classification": "HS",
  "frequency": "A",
  "flows": ["import"],
  "partner_code": "all",
  "commodity_codes": ["01", "02", "03", "04", "05"],
  "max_codes_per_batch": 20
}
```

The code should be written so that later we can expand to:

```
HS2 imports
HS2 exports
HS4 imports
HS4 exports
HS6 imports
HS6 exports
```

The script should be fault-tolerant. It should:

- clearly fail if the subscription key file is missing;
- clearly fail if `config/panel_config.json` is missing or invalid;
- warn if reference files are missing but still run using available API data;
- save empty CSVs only with a clear warning;
- handle API failures gracefully and continue to the next batch where possible;
- not expose the subscription key in printed output;
- not hard-code user-specific secrets;
- not change the ontology report.

The script must normalise likely Comtrade column names. Depending on API output, columns may vary, so implement a helper that identifies or maps columns such as:

```
period / refYear
reporterCode / reporterISO / reporterDesc
partnerCode / partnerISO / partnerDesc
cmdCode
cmdDesc
flowCode / flowDesc
primaryValue / TradeValue / CIFValue / FOBValue
netWgt / qty
```

Use `primaryValue` as the preferred trade value column where available. If not available, fall back to other value columns.

For dressing / cleaning:

- make sure year, reporter, partner, commodity, flow, and trade value fields are present;
- convert trade value to numeric;
- replace missing trade values with zero;
- aggregate duplicate rows by year, reporter, partner, commodity, and flow;
- create a complete panel from all configured years, partners, commodities, and flows;
- merge actual values into that complete panel;
- fill missing values with zero.

For wide output:

- create one wide table where rows are partner/year or partner/commodity/year as appropriate;
- columns should represent commodity-flow combinations where useful;
- filenames should be clear, for example:

```
india_import_long_panel.csv
india_import_wide_panel.csv
india_export_long_panel.csv
india_export_wide_panel.csv
```

or, if both flows are included:

```
india_trade_long_panel.csv
india_trade_wide_panel.csv
```

Add a top-level docstring to `comtrade_panel_builder.py` similar to this:

```
"""
UNCOMTRADE R7 Project Reconstruction

Ontology reference:
```

```
ontology/UNCOMTRADE_R7_ONTOLOGICAL_PROJECT_REPORT.md

This script recreates the old R7 UN Comtrade workflow according to the
ontology, data logic, and fidelity principles documented in the ontology
report. It replaces the legacy Comtrade URL procedure with the modern
UN Comtrade API while preserving the old project sequence:

download → dress / clean → transpose / wide panel

The subscription key is read from:
secrets/comtrade_primary_key.txt

The key must never be hard-coded in this script.
"""
```

Also create a small `README_RECREATED_WORKFLOW.md` in the project root explaining:

1. What this recreated project does.
2. Why the old R procedure no longer works.
3. Where the ontology report is located.
4. Where to place the subscription key.
5. How to run the script.
6. What output folders are created.
7. How to expand the config from the test run to larger HS2 / HS4 / HS6 runs.

The README should include these commands:

```
cd /media/ambijat/SOPRANO2/GPT_workflow/COMTRADE
source .venv/bin/activate
python comtrade_panel_builder.py
```

After implementing, run these checks:

```
python -m py_compile comtrade_panel_builder.py
python comtrade_panel_builder.py
```

If the API call cannot run because the subscription key or package is missing, do not fake success. Instead, make the error message precise and leave the script ready to run once the key/package is available.

At the end, report:

1. Which files were created or modified.
2. Whether the script compiled.
3. Whether the test run executed.
4. Any remaining dependency or API-key issue.
5. The next recommended expansion stage.

Do not erase existing project files. Preserve project fidelity.